

Enterprise Agentic Delivery

I start with one useful outcome.

I keep authority, evidence and recovery explicit.

I add coordination only when a measured gap requires it.

A control-plane product is optional. Its responsibilities may still be needed.

Remove the mandatory platform

Existing IAM, repository rules, builds, release controllers and telemetry can supply control functions.

Keep consequences explicit

For consequential work, define authority, exact-change evidence, durable progress and recovery.

I tested both sides of the argument.

A mandatory central plane

Disconfirmed by workflows already governed and recoverable through existing systems.

Harnesses remove all controls

Disconfirmed by stale approvals, uncertain external effects and conflicting workers.

A thin layer always helps

Reject it if it duplicates reliable existing state and costs more than it saves.

The harnesses offer complementary patterns.

Fleet and specialists

Cursor: direction separate from execution.

Copilot: explicit dependencies.

Claude: coding loop and bounded specialists.

Managed and missions

Codex: managed harness with environment choice.

Factory: planned milestones and validators.

Kiro / AWS: development-to-operations access.

Frameworks are alternatives, not layers to accumulate.

State and composition

ADK: agents with deterministic nodes.

LangGraph / Deep Agents: checkpoints and harness.

Microsoft Agent Framework: maintained successor to AutoGen.

Open and configurable

CrewAI: crews within flows.

OpenHands: modular software-agent runtime.

MetaGPT / ChatDev: explicit roles and configurable collaboration.

Three contracts keep the architecture small.

Outcome

What I want.

Owner, scope, constraints,
acceptance, risk and budget.

Evidence

What was established.

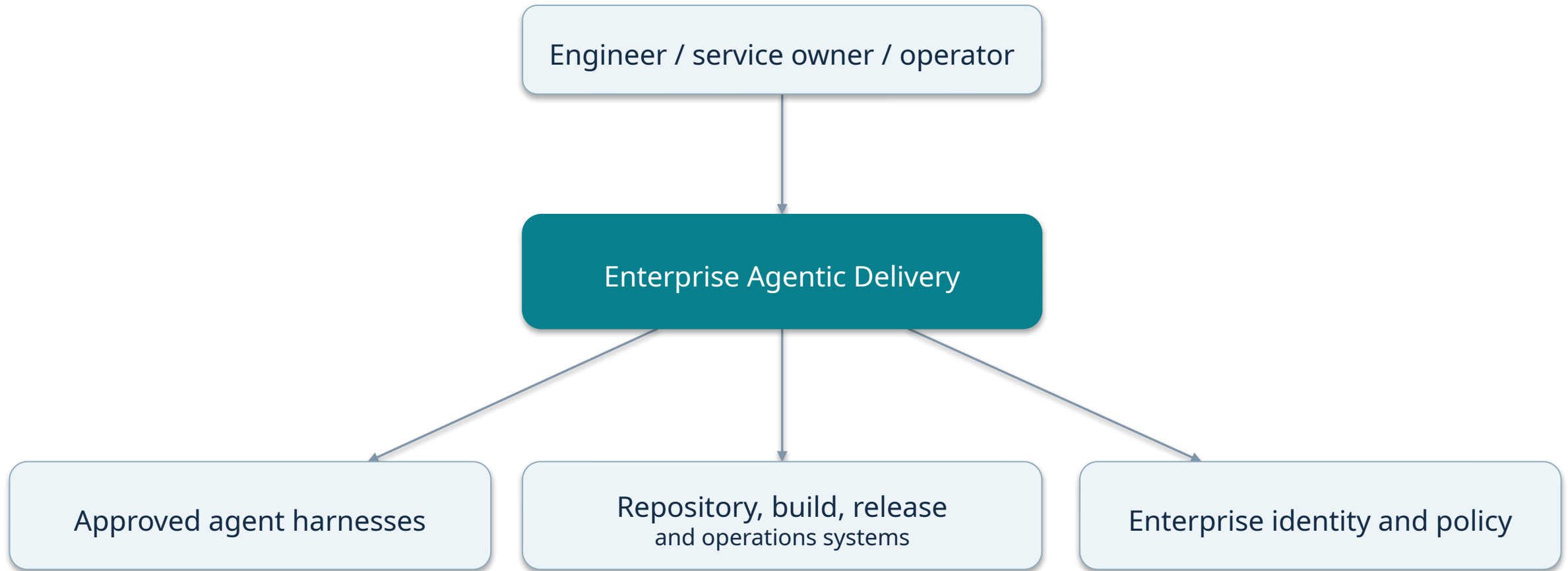
Exact revision/digest, trusted
producer, environment and
freshness.

Action

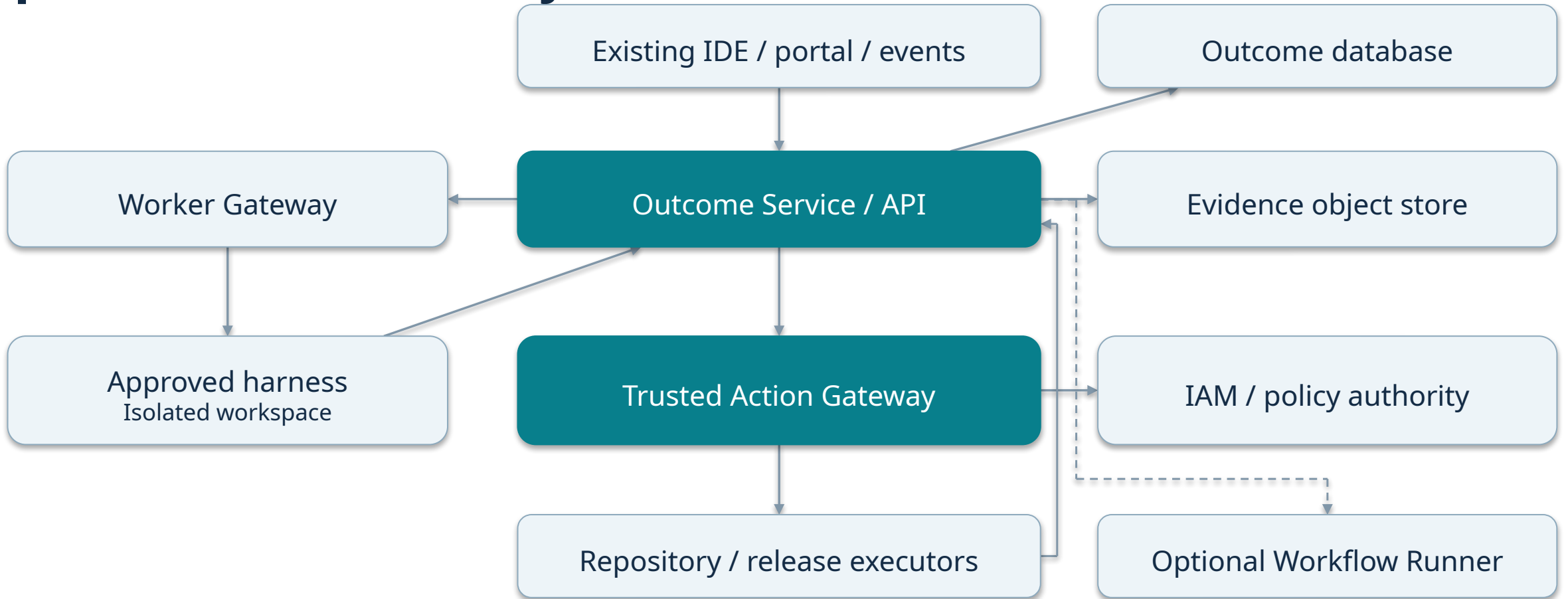
What may change.

Bounded effect, authority,
preconditions, operation
identity and recovery.

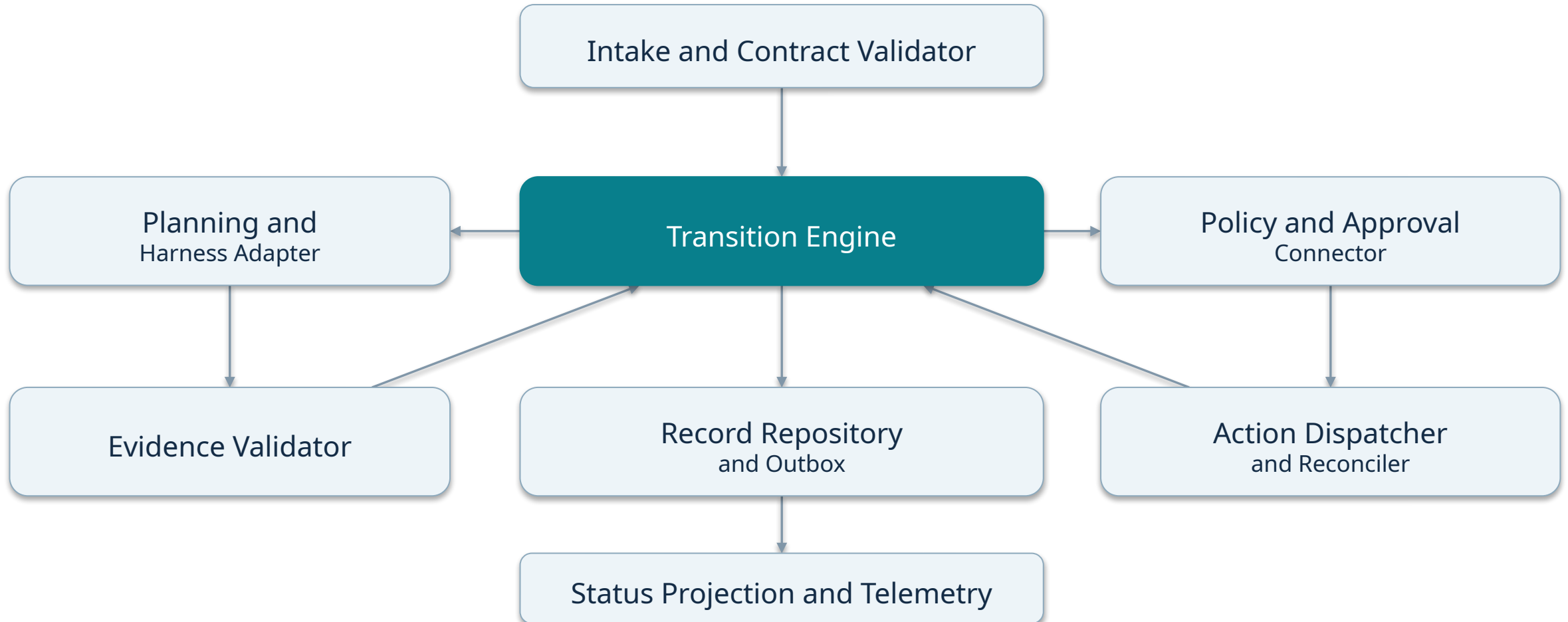
C1: the system fits into existing enterprise boundaries.



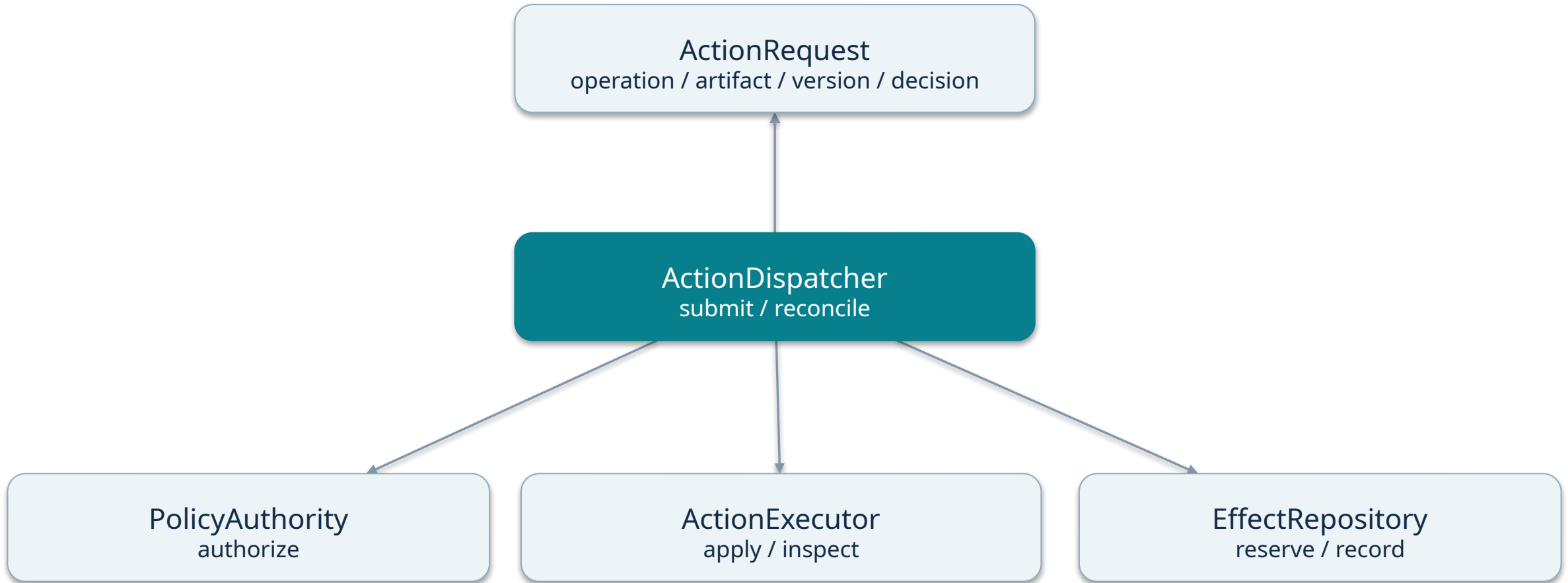
C2: reuse trusted executors; keep agents outside production authority.



C3: deterministic transitions surround flexible planning.



C4: the code boundary handles uncertainty explicitly.



Completion means the intended outcome was verified.

A paper completes when research and review criteria pass.

A PR completes when accepted behavior is demonstrated.

A release completes after its health contract passes.

An uncertain effect remains Indeterminate until reconciled.

Run useful checks early; retain independent enforcement.

At creation

Compile, test contracts, scan dependencies and apply policy before costly shared execution.

At acceptance

Bind trusted evidence to revision, artifact, suite, environment and policy. Reject stale or mismatched proof.

The hard cases are external effects and bypass paths.

Lost response

Inspect the executor operation before retrying. If the result is unknown, stop and reconcile.

Stale authority

Bind approvals to digest, target, operation, policy and expiry. Fence stale workers.

Prompt failure

Deny privileged paths outside the trusted executor. Treat retrieved instructions as untrusted content.

Memory improves work only if its learning can be evaluated and reversed.

Keep authoritative workflow state out of semantic memory.

Promote verified procedures with source, scope, owner and expiry.

Evaluate instruction changes on held-out cases.

Use a small cohort and retain the prior version.

Stages 0–2 already deliver useful results.

0 / Understand

Baseline one frequent outcome.

Identify tests, authority and actual waiting costs.

Use a script if sufficient.

1 / Improve creation

One harness, one delivery path.

Better accepted PRs and lower human effort.

Existing controls remain authoritative.

2 / Recover work

Add missing shared records and reconciliation.

Reduce interruption and coordination effort.

Stages 3–5 expand only after evidence supports them.

3 / Scale bounded work

Compare single-agent against owned parallel tasks.

Migrate in repository waves.

Stop if coupling erases benefits.

4 / Validate releases

One trusted executor and health contract.

Shadow first, bounded cohort next.

Test pause and recovery.

5 / Broaden outcomes

Investigate incidents before writes.

Add domain packs and governed learning.

Evaluate every new outcome family.

The same contracts support many enterprise outcomes.

Engineering

- Research and architecture
- Features and defect repair
- Dependencies and migrations
- Policy and pipeline remediation

Delivery and operations

- Validated release
- Incident investigation
- Bounded recovery
- Infrastructure and data changes

CI/CD products retire only after their functions are replaced safely.

Start with Jenkins, Spinnaker, GitLab or other existing executors.

Request contract-shaped outcomes through adapters.

Replace a tool only after equivalent controls, recovery and economics pass.

An agent loop does not remove the need for reproducible execution.

Measure accepted outcomes, total effort and service effects.

Useful measures

Accepted outcomes / all attempts

Validated completion time

Human effort and review rejection

Escaped defects and service regressions

Full cost

Model and compute

Integration and rework

Human review

Allocated platform operation

Divided by accepted outcomes

I would fund one pilot and preserve the right to simplify.

One frequent outcome. One approved harness. One trusted path.

Define evidence and authority before expanding the fleet.

Keep the custom layer only where it proves useful.

Progress by measured value and tested recovery.

Evidence limits are part of the recommendation.

Primary documentation reviewed on 6 October 2026.

No paid-product benchmark or production pilot was performed.

Product access and versions change; recheck before procurement.

Enterprise obligations, RTO/RPO and ROI remain deployment-specific.

Primary-source ledger

- [1] Cursor, Introducing Projects
- [2] GitHub, Fleet mode
- [3] Anthropic, Agent SDK overview
- [4] Anthropic, Claude Opus 4.6 announcement
- [5] OpenAI, Agents API architecture
- [6] OpenAI, Agents overview
- [7] Factory, Missions overview
- [8] Factory, Planning and Validation
- [9] Factory, Running in the Factory App
- [10] Kiro, Invoking as sub-agents
- [11] AWS, DevOps Agent expansion
- [12] Google ADK, Workflows
- [13] LangChain, LangGraph Persistence
- [14] LangChain, Deep Agents overview
- [15] Microsoft, AutoGen maintained repository

Primary-source ledger

[16] Microsoft, Agent Framework checkpoints

[17] CrewAI, Flows

[18] OpenHands, software-agent SDK

[19] OpenHands, delegation example

[20] FoundationAgents, MetaGPT

[21] OpenBMB, ChatDev

[22] C4 model, Diagrams

[23] MCP, Security Best Practices

[24] A2A, Specification

[25] SLSA, Provenance v1.1

[26] Temporal, Workflow Execution

[27] Factory, Missions Reference

[28] OpenAI, Agents API overview

[29] Cursor, Projects documentation